UTN FRLP

Desarrollo de Software

Integración de una aplicación con una base de datos NoSQL

Martin Forlini Simon

Profesores Emilio Watemberg y Giudici Paula

¿Qué es una Base de Datos No Relacional (NoSQL)?

A diferencia de las bases de datos relacionales (SQL) que utilizan tablas y filas rígidas, las **NoSQL** almacenan datos en formatos flexibles como documentos, grafos o pares clave-valor. No requieren un esquema fijo, lo que permite que cada registro tenga una estructura diferente si es necesario.

¿Cuándo es conveniente usarla?

- Cuando los datos son **no estructurados** o cambian constantemente.
- En proyectos que requieren una **alta escalabilidad** horizontal (crecer en múltiples servidores).
- Desarrollo ágil donde el modelo de datos evoluciona rápido
- Cuando no es necesario guardar una persistencia a largo plazo

Ventajas	Desventajas
Flexibilidad: No necesitas definir tablas de antemano.	**Consistencia:** Menos rígidas con la integridad referencial.
Escalabilidad: Manejan grandes volúmenes de datos fácilmente.	**Consultas complejas:** Más difíciles de realizar que en SQL.
Velocidad: Lectura y escritura rápida de registros simples.	**Duplicidad:** A veces se repiten datos para ganar velocidad.
Errores: Menores errores y de menores consecuencias	**Limitación: Puede ser limitada a largo plazo o para sistemas grande**

¿Cuáles existen?

- **Documentales:** MongoDB, CouchDB.
- **Clave-Valor:** Redis, DynamoDB.
- **Grafos:** Neo4j.
- **Columnares:** Cassandra.

2. Planteo de la Problemática y Elección de MongoDB

Problemática: Gestión de Comandas en Tiempo Real
aca poner sobre comandas

Un sistema de pizzería necesita gestionar pedidos que varían constantemente: una pizza puede tener notas especiales, diferentes variedades, o métodos de entrega distintos (Rappi, PedidsYa, Delivery Propio, Retiro en puerta). Una estructura de tabla rígida obligaría a dejar muchos campos vacíos o crear tablas relacionales complejas para algo que debe ser fluido.

¿Por qué es aplicable?

Las comandas son, por naturaleza, **documentos**. Un pedido es un objeto que contiene una lista de items, un cliente y un estado. No hay necesidad de "unir" (join) muchas tablas para ver qué pidió un usuario; toda la información vive dentro del mismo objeto "Ticket" para luego de unos minutos eliminarse

¿Por qué elegimos MongoDB?

1. **JSON Nativo:** MongoDB almacena datos en BSON (un JSON binario). Como nuestra app está en **React/JavaScript**, la comunicación es natural: lo que ves en el estado de React es lo que se guarda en la base.
2. **Esquema Flexible:** Si mañana decidimos agregar "Empleado que tomó el pedido" o "monto a pagar", no tenemos que migrar toda la base de datos; simplemente empezamos a guardar el nuevo campo.
3. **Velocidad de Desarrollo:** Permite pasar de la idea al prototipo sin perder tiempo diseñando diagramas de entidad-relación complejos.

3. Solucion Implementada

Sistema de Gestión de Pedidos

Aplicación full stack para gestionar comandas de una pizzería.
Permite crear pedidos, agregar múltiples pizzas, marcar prioridad, avanzar estados y eliminar pedidos.

Los estados del pedido son:
PENDIENTE → EN_PREPARACION → LISTO → DESPACHADO

La información se guarda en MongoDB, por lo que los datos persisten aunque se reinicie la aplicació

Funciones Implementadas:

CRUD completo

Persistencia real en MongoDB

Manejo de múltiples ítems por pedido

Avance de estados

Gestión de prioridad

Interfaz tipo tablero

Arquitectura de la solución

La aplicación sigue una **arquitectura cliente-servidor de tres capas**:

1. **Frontend**: React + Vite, ejecutándose en el navegador del usuario, responsable de la interfaz y la interacción.

2. **Backend**: Node.js + Express, expone una API REST que gestiona la lógica de negocios y recibe las solicitudes del frontend.

3. **Base de datos**: MongoDB, almacena de forma persistente los pedidos y sus detalles, accedida desde el backend mediante Mongoose.

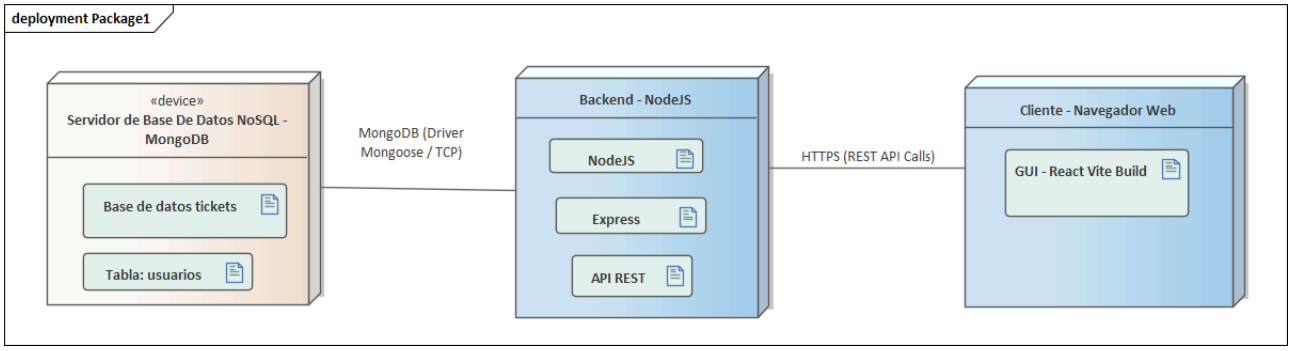
Tecnologías utilizadas

- **Frontend**: React, Vite, TypeScript, CSS.

- **Backend**: Node.js, Express, Mongoose.

- **Base de datos**: MongoDB (Community Server)

Diagrama de Despliegue



Dificultades Encontradas:

Sincronización de estados: mantener la UI actualizada con los cambios de la base de datos en tiempo real o al hacer operaciones CRUD.

Gestión de arrays anidados: agregar, quitar y actualizar múltiples ítems (pizzas) dentro de un mismo pedido de manera eficiente.

Validaciones de formulario: controlar que los campos obligatorios estén completos antes de enviar al backend (número de pedido, al menos una pizza).